

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1516

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I Y SRAVAN KUMAR (192472289) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____ SRAVAN Y _____

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood

System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

1. HDFS Storage Solution for a Media Platform

Problem

A media platform stores a large amount of **videos, images, audio, and metadata**. The solution must satisfy three constraints:

- Data availability must be **above 99.9%**
- Replication factor must **not exceed 3**
- Storage growth must remain **cost efficient**

Proposed HDFS Architecture

Media Files → HDFS Data Nodes → Replication (3 copies maximum) → Processing/Analytics

Design

- HDFS NameNode**
 - Manages file metadata and the location of blocks.
 - Use **NameNode High Availability** with Active and Standby NameNodes to improve availability.
- HDFS DataNodes**
 - Store actual media file blocks.
 - Data is distributed across multiple DataNodes.
- Replication Factor = 3**
 - Each important data block can have up to three copies.
 - Copies should be placed on different DataNodes and preferably different racks.
- Rack Awareness**
 - HDFS distributes replicas across racks.
 - If one rack fails, another replica remains available.

5. Storage Cost Control

- Use the replication factor of 3 only where required.
- Older or less frequently accessed media can be moved to cheaper storage tiers.
- Compress metadata and suitable media-related data.
- Increase storage nodes gradually as the data grows.

Why This Design Works

Constraint	Solution
>99.9% availability	NameNode HA + replicated DataNodes
Replication ≤ 3	Maximum replication factor of 3
Cost efficiency	Storage tiering + compression + gradual expansion
Node failure	Replicas available on other nodes
Large media files	Distributed HDFS blocks

Conclusion

The design provides high availability without exceeding the replication limit. **HDFS replication, rack awareness, NameNode HA, and controlled storage expansion** provide a balance between reliability and cost.

2. MapReduce Workflow Under a 20-Minute Batch Constraint

Problem

A system must process a large amount of **log data within 20 minutes**, but the number of cluster nodes is limited.

The main challenge is to achieve maximum parallelism with the available nodes.

Proposed Workflow

Log Files → HDFS → Map Phase → Shuffle & Sort → Reduce Phase → Output

Map Phase

The input logs are divided into blocks.

Each block is processed by a **Mapper task**.

For example:

Log Data

↓

Block 1 → Mapper 1

Block 2 → Mapper 2

Block 3 → Mapper 3

Block 4 → Mapper 4

The mappers can extract information such as:

- IP address
- URL
- Status code
- Timestamp
- Number of requests

Shuffle and Sort

The intermediate key-value pairs generated by mappers are grouped according to their keys.

Example:

Mapper Output

/home → 100

/home → 200

/login → 50

The shuffle phase groups similar keys together.

Reduce Phase

Reducers aggregate the grouped results.

For example:

/home → 300

/login → 50

Task Distribution

Since cluster nodes are limited:

- Divide input into appropriately sized blocks.
- Run multiple mapper tasks in parallel.
- Assign reducer tasks according to the number of keys and output volume.

- Prefer **data locality**, so computation runs close to the data.
- Avoid unnecessary data transfer.
- Use combiners where possible to reduce shuffle traffic.

Meeting the 20-Minute Constraint

To finish within 20 minutes:

1. Use parallel mapper execution.
2. Optimize block sizes.
3. Use combiners to reduce intermediate data.
4. Minimize unnecessary shuffle data.
5. Balance reducer workload.
6. Use efficient serialization and compression.
7. Monitor slow or failed tasks.

Example

If there are **8 available processing slots**, the input can be divided into several blocks and up to 8 mapper tasks can execute concurrently.

Conclusion

The workflow meets the 20-minute requirement by using **parallel Map tasks, data locality, efficient shuffle, combiners, and balanced Reduce tasks** while making maximum use of the limited cluster nodes.

3. YARN Scheduler for ETL, Reporting and Machine Learning

Problem

A YARN cluster needs to run three types of jobs:

- ETL
- Reporting
- Machine Learning

The scheduler must satisfy:

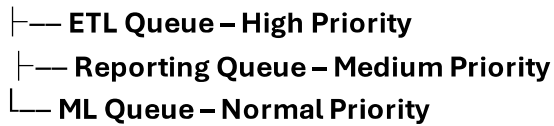
1. Fair resource sharing
2. Higher priority for ETL
3. Recovery after node failure

Proposed YARN Scheduler Policy

A **Capacity Scheduler** or appropriately configured YARN scheduler can organize resources into queues.

Queue Design

YARN Cluster



ETL Queue

ETL jobs receive higher priority because they prepare data required by downstream analytics.

For example:

ETL → Reporting → ML

The ETL queue can have a larger minimum resource allocation.

Reporting Queue

Reporting jobs receive a moderate allocation.

They should receive resources without completely blocking other workloads.

ML Queue

Machine learning jobs can use remaining cluster resources and can be scheduled according to available capacity.

Fairness

Fairness can be maintained by:

- Defining queue capacities
- Limiting maximum resources per queue
- Preventing one application from consuming the entire cluster
- Sharing unused resources with other queues

Node Failure Recovery

If a NodeManager fails:

1. YARN ResourceManager detects the failure.
2. Tasks running on the failed node are identified.
3. Failed tasks are rescheduled on healthy nodes.
4. Replicated input data is accessed from another DataNode.
5. The application continues processing.

Example Priority

Queue	Priority	Purpose
ETL	High	Data preparation
Reporting	Medium	Business reports
ML	Normal	Model training

Conclusion

A queue-based YARN policy provides **ETL priority while maintaining fairness**. YARN's distributed resource management and task rescheduling also provide recovery when nodes fail.

4. Enterprise Data Ingestion from SAN, NAS and Operational Databases

Problem

An enterprise wants to ingest data from:

- **SAN**
- **NAS**
- Operational databases

The architecture must provide **reliable backup** while minimizing duplicate data.

Proposed Architecture

SAN + NAS + Operational DBs

↓

Data Ingestion / ETL Layer

↓

Central Data Lake

↓

Processing & Analytics



Backup / Disaster Recovery

1. SAN

SAN provides block-level storage and can contain large enterprise datasets.

Data can be accessed through appropriate storage interfaces and ingestion tools.

2. NAS

NAS provides file-based storage.

Files can be transferred or ingested into the data lake using suitable connectors or ETL processes.

3. Operational Databases

Database connectors can extract:

- Customer records
- Orders
- Transactions
- Inventory
- Business data

4. ETL/ELT Layer

An ETL tool can:

- Extract data
- Validate data
- Remove duplicates
- Transform formats
- Load data into the Big Data platform

5. Deduplication

To minimize duplication:

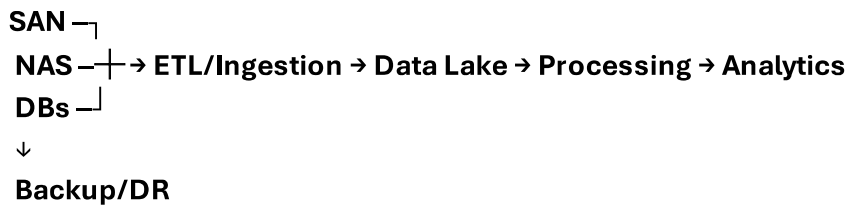
- Use unique record IDs.
- Compare source and target records.
- Use checksums/hashes for files.
- Use incremental ingestion instead of repeatedly copying complete datasets.
- Maintain metadata about already ingested data.

6. Backup Reliability

Important data should have:

- Regular backups
- Multiple copies
- Separate backup storage
- Recovery testing

Architecture



Why This Design Works

Requirement	Solution
SAN ingestion	Storage connector/ETL
NAS ingestion	File-based ingestion
Database ingestion	DB connectors
Minimal duplication	Incremental loading + deduplication
Backup reliability	Replication + separate backup
Central analysis	Data Lake

Conclusion

The architecture provides a **centralized and reliable data platform** while reducing unnecessary duplication through incremental ingestion, metadata tracking, and deduplication.

5. End-to-End Hadoop Ecosystem with Security and High Availability

Problem

An organization needs an end-to-end Hadoop ecosystem that supports:

- Secure data movement
- High availability
- Apache NiFi or ETL integration

Proposed Architecture

Data Sources

↓

Apache NiFi / ETL

↓

Secure Data Transfer

↓

Kafka / HDFS

↓

YARN + Spark/MapReduce

↓

Hive / HBase / Analytics

↓

Dashboard

1. Data Ingestion Using Apache NiFi

Apache NiFi can collect data from:

- Databases
- Files
- Applications
- IoT systems
- External services

NiFi can also provide data routing, transformation, and monitoring.

2. Secure Data Movement

Security can be provided using:

- TLS/SSL encryption
- Authentication
- Authorization
- Kerberos in Hadoop environments

- Access control

Sensitive data should be encrypted while moving between systems.

3. HDFS Storage

HDFS stores large datasets across multiple DataNodes.

Replication provides protection against individual node failures.

4. High Availability

The architecture can use:

- Active/Standby NameNodes
- Multiple DataNodes
- Replication
- ResourceManager HA
- Multiple processing nodes

If one important service fails, another instance can take over.

5. Processing

YARN manages cluster resources.

Processing can be performed using:

- Apache Spark
- MapReduce

Spark is useful for faster large-scale processing, while MapReduce can handle traditional batch workloads.

6. Data Analysis

Hive can provide SQL-like analysis over Hadoop data.

HBase can be used when fast random read/write access to large datasets is required.

7. Overall Architecture

Data Sources

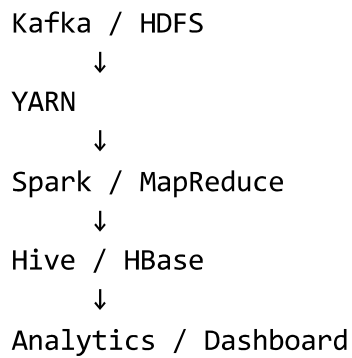
↓

Apache NiFi / ETL

↓

Secure Data Movement

↓



How Constraints Are Satisfied

Constraint	Design
Secure data movement	TLS/SSL + authentication + authorization
High availability	NameNode HA + ResourceManager HA
Storage reliability	HDFS replication
Data ingestion	Apache NiFi / ETL
Large-scale processing	Spark / MapReduce
Resource management	YARN
Data analysis	Hive / HBase

Conclusion

The proposed Hadoop ecosystem provides **secure ingestion, reliable distributed storage, high availability, scalable processing, and integration with Apache NiFi/ETL pipelines.**